

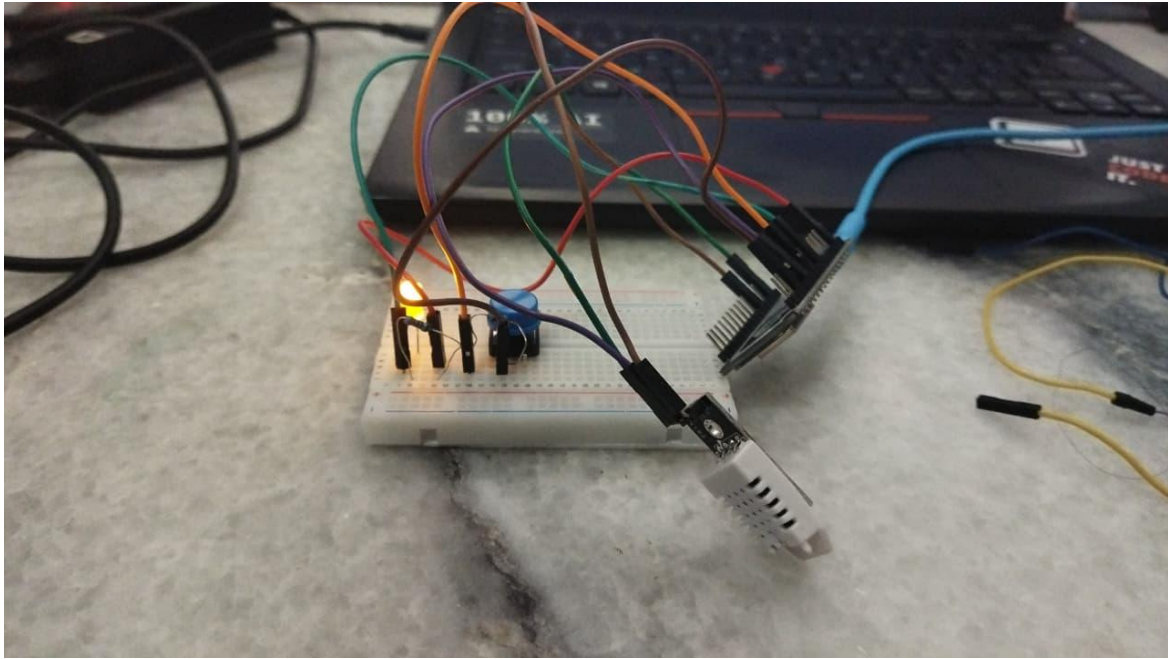
4.5 Praktikum

4.5.1 Praktikum 1: Dashboard & Sinkronisasi Tombol Fisik *Real-Time*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang dapat dihidupkan melalui saklar fisik (*push button*) pada *breadboard* , ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time* . Jika Anda menekan tombol fisik di meja praktikum, indikator visual status di antarmuka *ponsel cerdas* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Penjelasan Singkat Kode Khusus:

- `ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/ *browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berakhir pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard* .
- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*beralih*) LED secara instan.
- `if ((millis() - lastTime) > 3000)`: Pemrograman *non-blocking* . Alih-alih membuat CPU tertidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respon WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga memperhatikan sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- `var dataObj = JSON.parse(event.data)`: Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat `{"suhu": "30", "led": "1"}` menjadi struktur data objek dinamis (*Kamus*) untuk memperbarui elemen desain HTML dengan instan (*Manipulasi DOM*).
- `ws.textAll()`: Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.



⚠ Not secure 192.168.30.173



Smart Room

Suhu: 31.70

LED: ON

Turn OFF

4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- **Latihan 1 (Teori Jaringan):** menjelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

JAWAB : *WebSockets* lebih hemat bandwidth karena koneksi dibuka satu kali dan tetap terbuka (*persistent*), lalu hanya mengirim paket data JSON kecil saat ada perubahan. *Meta Refresh* harus meminta ulang seluruh halaman HTML (beserta CSS dan header HTTP) setiap kali refresh, padahal yang berubah hanya satu angka.

WebSockets juga tidak membuat layar berkedip karena halaman hanya dimuat sekali. Saat data baru datang, JavaScript hanya mengganti teks pada elemen tertentu (*DOM manipulation*). Pada *Meta Refresh*, seluruh halaman dimuat ulang sehingga layar tampak berkedip (*flickering*).

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda mengizinkan nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

JAWAB : Bahaya `delay(3000)` di `loop()`

`delay(3000)` menghentikan CPU selama 3 detik di setiap siklus `loop()`, sehingga:

1. Tombol fisik tidak responsif. Penekanan yang terjadi saat `delay` berjalan tidak terbaca.
2. Sinkronisasi dua arah tertunda. Perubahan dari tombol fisik terlambat dikirim ke browser, dan perintah dari browser terlambat dieksekusi ke LED.
3. Koneksi *WebSocket* tidak stabil. Tugas jaringan dan `ws.cleanupClients()` jarang berjalan, sehingga klien bisa terputus.
4. Sifat *real-time* hilang.

Solusinya adalah memakai `millis()` (*non-blocking*) agar CPU tetap bisa memantau tombol dan jaringan.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

JAWAB : Target Injeksi Teks JavaScript

Elemen `<span>` yang menjadi target perubahan teks:

- `tempValue`: menampilkan nilai suhu.
- `ledStatus`: menampilkan status LED ("ON" / "OFF"). Ini satu-satunya `<span>` yang diubah pada blok *Merombak UI Tombol & Teks*.

Selain itu, blok tersebut juga mengubah teks dan warna (kelas CSS) tombol `toggleBtn`.

- **Latihan 4 (Pengembangan Sensor Telemetry - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperatur*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* melalui WebSockets:
 - **Sisi Back-End (C++):** Untuk penampung variabel (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` dalam siklus timer *non-blocking* `millis()` . memperbarui fungsi `notifyClients()` agar menyisipkan kelembapan data ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
 - **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak kelembapan data dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

